

Informe Técnico Red Team — Ronda 1

Proyecto: Operación Centinela · Cliente: NovaPay · Fecha: 25 de mayo de 2026
De: Equipo Red Team (Ciberseguridad) · Para: Equipo Data Science (Blue Team)

1. Contexto y alcance

Durante la Ronda 1 hemos evaluado la resistencia del modelo XGBoost expuesto en `/predict/` contra cuatro tipologías de fraude diseñadas con conocimiento *gray-box* (feature importance y schema OpenAPI compartidos por vuestro equipo). En total se han evaluado **170 transacciones** sobre IDs reales del pool. El objetivo no era romper el modelo, sino mapear qué patrones aprende y dónde están sus puntos ciegos antes de la Ronda 2. Las cuatro tipologías son complementarias: **T1** y **T2** prueban fraude *mimético* (atacante que se comporta como usuario legítimo); **T3** mide el peso aislado de cada feature; **T4** mide qué combinaciones disparan el clasificador y si existen categorías comerciales explotables como 'zonas seguras'.

2. Tipologías evaluadas

T1 — Suplantación silenciosa (Account Takeover camuflado)

Un atacante consigue las credenciales de un usuario real (phishing, filtración, malware) y, en lugar de hacer compras llamativas, replica el comportamiento del titular: importes en su rango habitual (€15–€400), comercios cotidianos, país habitual, móvil, sin VPN, sin prisas. ¿Distingue el modelo al titular real del impostor cuando ambos hacen lo mismo?

Métrica	Resultado
Transacciones procesadas	57 (60 enviadas, 3 bloqueos por rate-limit)
Tasa de evasión (score < 30%)	100% (57/57)
Score medio / máximo	0,91% / 27,6%
Transacciones en banda de revisión o fraude	0

T2 — Microtransacciones de validación (Card Testing)

Un atacante con un lote de tarjetas robadas hace ráfagas de microcompras de €0,50–€8 (suscripciones, productos baratos) separadas por minutos para saber qué tarjetas siguen activas antes de usarlas en compras grandes. ¿Detecta el modelo el patrón de ráfaga?

Métrica	Resultado
Transacciones procesadas	39 (40 enviadas, 1 bloqueo por rate-limit)
Tasa de evasión	100% (39/39)
Score medio / máximo	4,40% / 27,8%
Velocity declarada alta (1–5min) vs normal (30–300min)	4,88% vs 3,93% · $\Delta = 0,95$ pp

T3 — Ablación de features individuales

Partiendo de una transacción base normal (€30, perfil legítimo), modificamos una sola variable a la vez para medir cuánto peso tiene cada señal de riesgo por sí sola.

Variante alterada	Score medio	Resultado
uso_vpn_proxy = 1	2,3%	Evadido
tipo_tarjeta = Prepago	1,2%	Evadido
mismo_envio_facturacion = 0	0,8%	Evadido
Categoría = Electrónica, importe = €2.000	21,4% (pico 68,2%)	Frontera
pais_pago = RU	71,9%	Bloqueado
es_online = 1 + Desktop	4,8%	Evadido

T4 — Combinaciones adversariales y anomalía categórica

Probamos combinaciones de varios factores de riesgo a la vez para identificar el patrón mínimo que dispara la detección; en paralelo, buscamos categorías comerciales donde el modelo asuma seguridad por defecto.

Combinación	n	Resultado
País no-ES + Desktop + es_online + Suscripciones/Electrónica	10	80% detectado ($\geq 50\%$)
uso_vpn_proxy = 1 + pais_pago = RU	1	100% (bloqueo inmediato)
Supermercado, importe €3.000, perfil legítimo	3	$\leq 9,9\%$ (evadido) — anomalía

minutos_desde_ultima_tx = 1 (transacciones idénticas consecutivas)	2	0,2% (evadido)
--	---	----------------

3. Cinco hallazgos sobre el modelo

1. Diagnóstico raíz: el modelo aprendió correlaciones marginales, no condicionales.

Esta es la lectura que unifica todo lo demás. El XGBoost aprendió señales del tipo "RU es de riesgo", "VPN+RU es de riesgo", "hora_madrugada es de riesgo", pero no aprendió condicionalidades del tipo "€3.000 es de riesgo dado que este usuario nunca ha gastado más de €100 en supermercado". Los cuatro hallazgos siguientes son corolarios de este.

2. Ceguera completa al fraude mimético sobre cuentas legítimas (T1).

Sin features que comparen la transacción actual con el historial del usuario (varianza de importe habitual, deriva de geolocalización, frecuencia por categoría, horario típico), el modelo es **estructuralmente incapaz** de distinguir al titular del atacante que se comporta como el titular. 57 de 57 evasiones en T1, score medio 0,91%.

3. Velocity no es una señal aprendida (T2).

El campo minutos_desde_ultima_tx no tiene peso predictivo relevante: la diferencia entre ráfagas (1–5 min) y velocity normal (30–300 min) es de solo 0,95 pp. Importante: todo el análisis se hizo declarando el valor del campo en el payload; el modelo no observa el timing real entre llamadas, solo lo que diga el JSON enviado.

4. Sesgo categórico explotable: Supermercado.

Importes hasta 30× la mediana habitual de la categoría no levantan el score por encima del 10%. El modelo trata Supermercado como categoría segura sin condicionar por importe ni por desviación del histórico del usuario. **Vector de explotación directo:** carding de volumen camuflado en comercios de baja sospecha.

5. La feature top-1 declarada (hora_madrugada, 15,6%) se neutraliza con coste cero.

Es una feature server-side no inyectable, pero el atacante simplemente evita atacar entre las 0h y las 5h y la neutraliza. En la práctica solo penaliza al atacante torpe. Hay además un segundo disparador robusto: **pais_pago = RU** activa por sí solo el bloqueo (score 71,9% en aislado, 100% en combinación con VPN), lo que es bueno frente a fraude burdo pero confirma que el modelo se apoya en correlaciones geopolíticas marginales en lugar de en patrones de comportamiento.

4. Recomendaciones para Ronda 2

Las siguientes acciones están ordenadas por impacto esperado sobre la detección, no por dificultad de implementación. Recomendamos al menos atacar 4.1, 4.2 y 4.5 antes de la Ronda 2, ya que cubren directamente las tipologías que evadieron al 100% en Ronda 1.

4.1 Features de comportamiento histórico por usuario (prioridad crítica — cierra T1 y la anomalía Supermercado)

El modelo no puede detectar fraude mimético porque no sabe nada de la historia del usuario al que está evaluando. Hay que enriquecer el input con agregados precomputados por id_usuario y joinarlos en el backend igual que ya se hace con la tabla clientes. Concretamente sugerimos calcular, sobre ventanas de 30 y 90 días:

- **Importe:** mean_amount_30d, std_amount_30d, p95_amount_30d, y derivar en tiempo real $z_score_current_amount = (importe - mean) / std$. Esto, por sí solo, habría disparado la anomalía Supermercado de €3.000.
- **Categoría:** top3_categories_30d (one-hot), is_current_category_in_top3 (binaria), n_tx_in_current_category_30d.
- **Geografía:** most_common_country_90d, is_current_country_new (binaria), days_since_first_tx_in_country.
- **Dispositivo:** n_distinct_devices_30d, is_current_device_new.
- **Horario:** mean_hour_of_tx_30d, std_hour_30d, is_current_hour_outlier ($|z| > 2$).

Implementación sugerida: job batch nocturno en Spark/Pandas que recompute los agregados por usuario y los persista en una tabla user_behavior_features. El endpoint /predict/ hace LEFT JOIN sobre esa tabla antes de inferir. Para usuarios nuevos sin historial suficiente (<5 transacciones), fallback a la media/std global de su segmento demográfico, marcado con un flag is_cold_start para que el modelo lo trate diferente.

4.2 Features de velocity reales (no declaradas) (prioridad alta — cierra T2)

El campo minutos_desde_ultima_tx tal como está hoy es engañoso: el atacante lo declara y el modelo lo cree. Hay que computar la velocity server-side a partir de un store transaccional consultable con latencia baja (Valkey/Redis encaja perfectamente). Features a exponer al modelo:

- n_tx_1h, n_tx_24h, n_tx_7d — conteo de transacciones del mismo id_usuario en ventanas móviles.
- sum_amount_1h, sum_amount_24h — suma de importes en ventana (detecta drenaje).
- n_distinct_merchants_1h, n_distinct_devices_1h — diversidad atípica en ventana corta.
- real_seconds_since_last_tx — calculado en backend desde timestamp de servidor, no declarado en payload. Reemplaza al campo actual.

Nota técnica: con TTL apropiado en Valkey (7 días para n_tx_7d, etc.) y un sorted-set por id_usuario indexado por timestamp, las consultas se resuelven en <5 ms y no penalizan la latencia del endpoint.

4.3 Reentrenamiento con fraude mimético sintético (prioridad alta)

El training set actual contiene fraude obvio (importes raros, países raros, VPN), por eso el modelo aprendió esas correlaciones marginales. Para que aprenda condicionalidades, hay que mostrarle ejemplos de fraude camuflado. Pipeline sugerido:

- Tomar un subset estratificado de transacciones legítimas del training (≈10k filas).
- Aplicarles una transformación que conserve el payload pero cambie el *actor implícito*: perturbar ligeramente importe ($\pm 20\%$ sobre el rango habitual del usuario), reemplazar el dispositivo por uno nunca usado por ese `id_usuario`, opcionalmente cambiar la hora a una compatible. Mantener categoría y país habituales.
- Etiquetarlas como fraude (`label=1`) y reincorporar al training set con un peso (`sample_weight`) calibrado para que representen ≈3% del set final.
- Reentrenar XGBoost con `scale_pos_weight` ajustado al nuevo balance (actual ~12% fraude → nuevo ~15%) y validar con stratified k-fold ($k=5$) reportando **precision/recall por tipología**, no solo AUC global.

4.4 Regularizar el peso de hora_madrugada

Mantener la feature pero reducir su capacidad de dominar el árbol. Tres opciones, de menos a más invasiva:

- **Opción A (rápida)**: aumentar `min_child_weight` y reducir `max_depth` del XGBoost. Penaliza splits superficiales que dependen demasiado de una sola feature.
- **Opción B (recomendada)**: sustituir la feature por una *interacción* explícita: `hora_madrugada × is_current_hour_outlier × z_score_amount`. Solo se activa cuando la hora rara coincide con un comportamiento raro del usuario, lo que cierra el agujero de evasión por simple desplazamiento horario.
- **Opción C (drástica)**: eliminarla y forzar al modelo a aprender otras señales. Solo si A y B no rinden.

4.5 Importe relativo al usuario, no absoluto (*prioridad crítica — cierra la fuga de Supermercado*)

El input actual contiene importe en valor absoluto. El modelo aprendió que €3.000 en Electrónica es sospechoso pero no lo aprendió en Supermercado, porque en el training casi nadie gasta €3.000 en supermercado *de forma fraudulenta* (sí lo hace de forma legítima, para hostelería, eventos, etc.). Solución:

- Añadir `amount_zscore_user_cat = (importe - mean[user, cat]) / std[user, cat]`, donde la media y la desviación se calculan sobre el histórico del *mismo usuario en la misma categoría* en los últimos 90 días.
- Añadir `amount_percentile_category_global` como respaldo para cold-start.
- Mantener importe absoluto en el payload, pero dejar que el modelo decida qué usa. En la práctica, los z-scores condicionales acabarán dominando los splits relevantes.

4.6 Monitorización post-deploy

Una vez aplicados los cambios, antes de la Ronda 2 conviene tener instrumentación lista para que el Red Team os pueda dar feedback objetivo. Mínimo recomendado: (a) **shadow scoring** del modelo nuevo en paralelo al actual durante 24h para comparar distribuciones de score sin riesgo de producción; (b) dashboard con precision/recall **desagregado por tipología** de ataque y por categoría comercial; (c) alertas de *data drift* en las nuevas features de comportamiento histórico ($PSI > 0,2$ sobre ventana móvil).

5. Nota técnica adicional para el equipo de backend

Durante el probing detectamos un comportamiento del API que **no es atribuible al modelo** pero sí afecta a la operación y conviene que lo conozcáis: el envío de un `id_usuario` no presente en la tabla `clientes` provoca una respuesta de `score = 1.0` o, en algunos casos, un volcado de error de SQLAlchemy relativo a tipos `np.float32`. Es decir, el endpoint no valida la entrada antes de intentar el join, y bajo ciertas condiciones expone trazas de la pila. Sugerimos validación explícita de `id_usuario` contra la tabla `clientes` al inicio del handler, devolviendo HTTP 400 con un mensaje genérico en caso de no existencia. Esto no es una vulnerabilidad explotable directamente para evadir detección, pero ensucia las métricas y filtra información estructural del backend.

Alcance del probing: 170 transacciones sobre IDs reales del pool, distribuidas entre las cuatro tipologías. Quedamos a disposición del equipo de Data Science para clarificaciones, datos brutos por transacción o una sesión conjunta de revisión antes del inicio de la Ronda 2.